

Ubiquitous Sensing, Computing & Communication Laboratory

Quality Laboratory Manual

Experiment No. 07

To design a system to detect seismic activity using Accelerometer and Gyroscope using Raspberry Pi.

Course Instructor – Mr.
Samish N Kamble
ASSISTANT PROFESSOR

Experiment No. 07

Title of Experiment: To design a system to detect seismic activity using Accelerometer and Gyroscope using Raspberry Pi.

Aim of Experiment: To design and implement a seismic activity detection system using an accelerometer and gyroscope (IMU sensor) interfaced with a Raspberry Pi, capable of detecting and analyzing vibrations and ground motion for early earthquake detection or structural monitoring.

System Requirements – Raspberry pi 4 Board Win 10 and above OS, 4GB RAM, 2.33 GHz Processor

Software/s Needed for Experiment –

Experiment Objectives:

1. To understand the working principles of accelerometers and gyroscopes (e.g., MPU6050).
2. To interface the IMU sensor with Raspberry Pi via I2C communication.
3. To measure real-time motion data (acceleration and angular velocity) using Python.
4. To analyze vibration patterns that may indicate seismic activity.
5. To alert users through a buzzer, LED, or display when unusual vibration thresholds are crossed.

Experiment Outcomes:

1. Understand and apply the principles of motion sensing for real-time monitoring.
2. Interface and collect data from an accelerometer and gyroscope using Raspberry Pi.
3. Implement a Python program to detect unusual seismic activity based on motion thresholds.
4. Design an alert system for early vibration warnings.
5. Gain practical experience in embedded systems for real-world geophysical monitoring applications.

1.Theory:

Seismic activity such as earthquakes or ground vibrations can be detected using motion-sensing devices like **accelerometers** and **gyroscopes**. These sensors are capable of detecting linear and angular movements, which can be used to sense tremors or abnormal vibrations in the Earth's surface or structures. In this experiment, we use a **Raspberry Pi** and an **IMU (Inertial Measurement Unit) sensor**—specifically an **MPU6050**, which includes a **3-axis accelerometer** and a **3-axis gyroscope**. The sensor continuously measures acceleration and rotation, and the Raspberry Pi processes this data to identify patterns that indicate potential seismic activity.

Accelerometer and Gyroscope Basics



Accelerometer:

- Measures **linear acceleration** in X, Y, and Z axes.
- Can detect movements such as shaking, tilting, or falling.
- Useful in detecting **vibrations or tremors** during seismic events.



Gyroscope:

- Measures **angular velocity** (rotation) in X, Y, and Z axes.
- Complements the accelerometer by detecting changes in orientation.
- Helps filter out noise or false positives in vibration data.

Role of Raspberry Pi

Raspberry Pi acts as:

- A **data acquisition** system that reads sensor values via **I2C protocol**.
- A **processor** that analyzes motion patterns to determine if they exceed a predefined vibration threshold.
- An **alerting device** that activates LEDs, buzzers, or even sends messages when abnormal motion is detected.
-

2. Wiring Connections:

Interfacing a gas sensor like the MQ-2 with a Raspberry Pi Pico involves connecting the sensor's analog output to a GPIO pin on the Pico and then reading the signal using the Pico's analog-to-digital converter

(ADC). A program, typically in MicroPython, is then written to read the ADC value and interpret it to determine gas presence or concentration.

1. Connecting the Gas Sensor:

- **Power and Ground:**

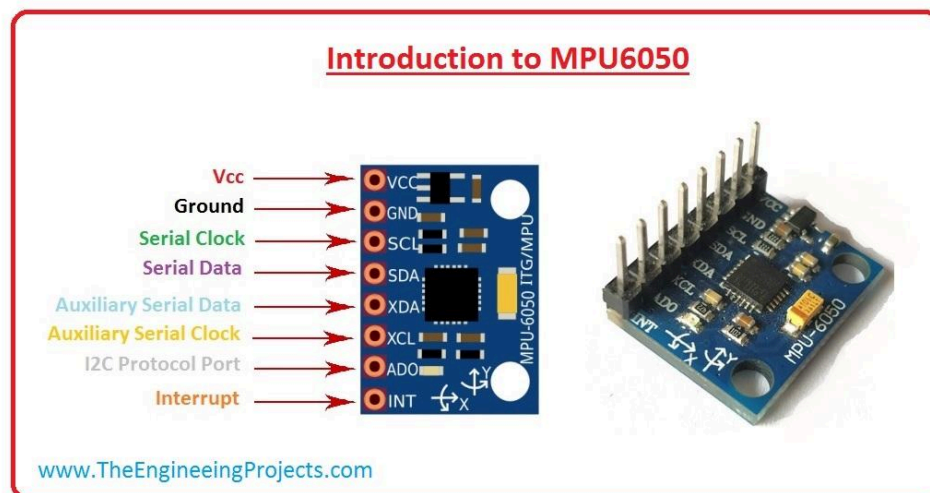
Connect the gas sensor's VCC to the Pico's 3.3V pin and the ground pin to the Pico's ground.

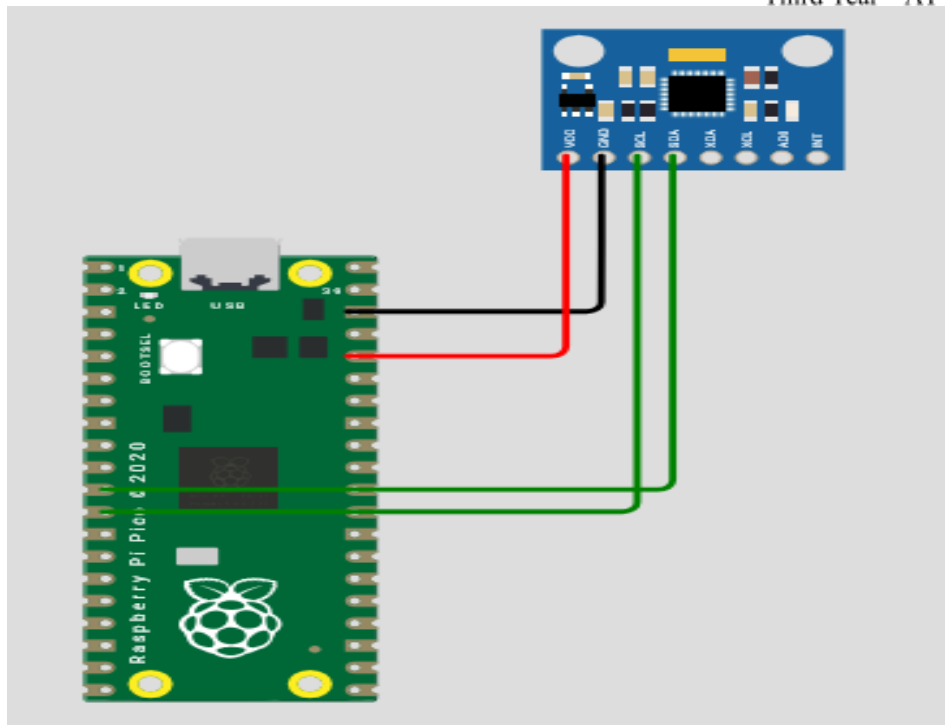
- **Analog Output:**

Connect the gas sensor's analog output to one of the Pico's GPIO pins that can be used for ADC readings (e.g., GPIO 26).

- **Optional:**

You may need a voltage divider circuit to scale the sensor's output voltage to the Pico's ADC input range if necessary.





3. Algorithm

Start the Program

Define Required MPU6050 Registers

- Set addresses for power management, accelerometer, gyroscope, and temperature registers.

Initialize I2C Communication

- Use I2C0 with specified SCL and SDA pins (Pin 9 and Pin 8).
- Set communication frequency (e.g., 400kHz).

Define Function to Write a Byte to a Register

- Used to send commands to MPU6050 (e.g., to wake it up).
- **Define Function to Read a 16-bit Signed Word from a Register**
- Read two bytes from a specified register and convert to signed integer.

Initialize MPU6050 Sensor

Wake up the MPU6050 by writing `0x00` to `PWR_MGMT_1` register.

Read Accelerometer Data

Read X, Y, and Z-axis acceleration values.

Read Gyroscope Data

Read X, Y, and Z-axis angular velocity values.

Read Temperature

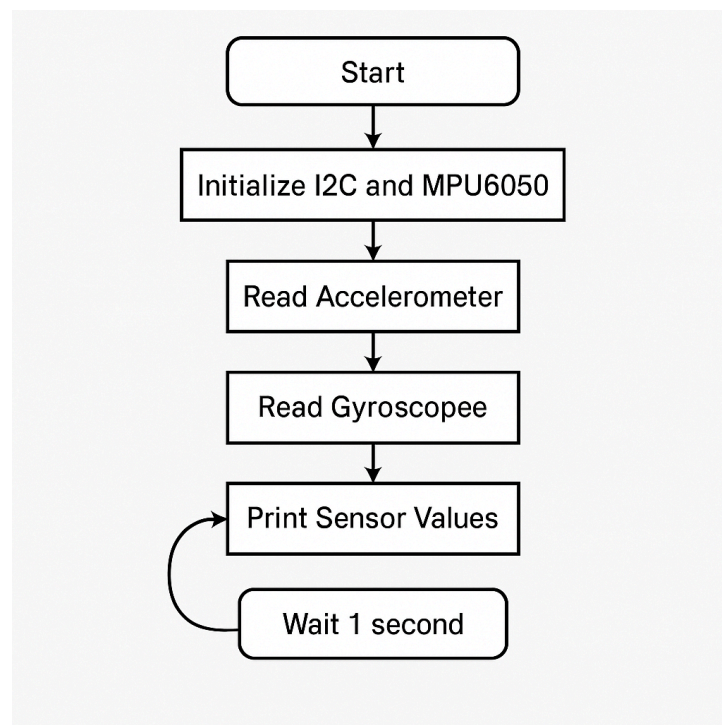
Read temperature register, convert raw data to Celsius.

Main Loop Begins

- Repeatedly perform the following steps:
 - a. Read accelerometer values
 - b. Read gyroscope values
 - c. Read temperature value
 - d. Print all sensor readings to console/serial monitor
 - e. Wait for 1 second

Repeat Step 10 Indefinitely

End

4. Flowchart

5. Program

```
from machine import Pin, I2C
import time
# MPU6050 registers
MPU6050_ADDR = 0x68
PWR_MGMT_1 = 0x6B
TEMP_OUT_H = 0x41
ACCEL_XOUT_H = 0x3B
GYRO_XOUT_H = 0x43
# Function to initialize I2C communication
def init_i2c():
    i2c = I2C(0, scl=Pin(9), sda=Pin(8), freq=400000) # Pico's I2C0
    return i2c
# Function to write a byte to a register
def write_byte(i2c, addr, reg, data):
    i2c.writeto_mem(addr, reg, bytearray([data]))
# Function to read a 16-bit signed value from a register
def read_word(i2c, addr, reg):
    data = i2c.readfrom_mem(addr, reg, 2)
    value = (data[0] << 8) | data[1]
    return value if value < 0x8000 else value - 0x10000 # Convert to signed
# Function to initialize MPU6050
def init_mpu6050(i2c):
    write_byte(i2c, MPU6050_ADDR, PWR_MGMT_1, 0) # Wake up the MPU6050

# Function to read accelerometer values
def read_accel(i2c):
    accel_x = read_word(i2c, MPU6050_ADDR, ACCEL_XOUT_H)
    accel_y = read_word(i2c, MPU6050_ADDR, ACCEL_XOUT_H + 2)
    accel_z = read_word(i2c, MPU6050_ADDR, ACCEL_XOUT_H + 4)
    return accel_x, accel_y, accel_z

# Function to read gyroscope values
def read_gyro(i2c):
    gyro_x = read_word(i2c, MPU6050_ADDR, GYRO_XOUT_H)
    gyro_y = read_word(i2c, MPU6050_ADDR, GYRO_XOUT_H + 2)
    gyro_z = read_word(i2c, MPU6050_ADDR, GYRO_XOUT_H + 4)
    return gyro_x, gyro_y, gyro_z

# Function to read temperature
```

```
def read_temp(i2c):
    temp_raw = read_word(i2c, MPU6050_ADDR, TEMP_OUT_H)
    temp_celsius = (temp_raw / 340.0) + 36.53 # MPU6050 temperature conversion
    return temp_celsius

# Main loop
def main():
    i2c = init_i2c()
    init_mpu6050(i2c)
    while True:
        accel_x, accel_y, accel_z = read_accel(i2c)
        gyro_x, gyro_y, gyro_z = read_gyro(i2c)
        temp_celsius = read_temp(i2c)
        print("Accelerometer X:", accel_x, " Y:", accel_y, " Z:", accel_z)
        print("Gyroscope X:", gyro_x, " Y:", gyro_y, " Z:", gyro_z)
        print("Temperature:", temp_celsius, "C")
        time.sleep(1)

if __name__ == "__main__":
    main()
```

Conclusion:

In this experiment, a **seismic activity detection system** was successfully developed using a **Raspberry Pi** and an **MPU6050 sensor** (accelerometer + gyroscope). The system continuously monitors acceleration and angular motion, enabling real-time detection of abnormal vibrations that could indicate potential seismic activity.

Expected Oral Questions:

1. What is the role of the MPU6050 sensor in your project?
2. How does the accelerometer differ from the gyroscope in terms of the data they provide?
3. Why is I2C communication used in this experiment, and how is it implemented in the code?
4. How can this system detect seismic activity using raw accelerometer and gyroscope values?
5. What improvements can be made to enhance the reliability and accuracy of this system?

FAQs in Interview:

1. Can you explain how the MPU6050 sensor works and why it's suitable for seismic activity detection?
2. How does your system differentiate between normal vibration and potential seismic activity?
3. Why did you choose Raspberry Pi (or Pico) for this project over other microcontrollers?
4. How would you scale this project into a real-time earthquake alert system using IoT?
5. What are some potential sources of error in your system, and how can they be minimized?